AI JOB MATCHER

A GRADUATION PROJECT SUBMITTED TO
THE FACULTY OF ENGINEERING AND ARCHITECTURE
OF
UNIVERSITY OF NEW YORK TIRANA

BY

Andrea Subashi

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF BACHELOR OF SCIENCE
IN
COMPUTER SCIENCE

JUNE 2025

APPROVAL

This is to certify that I have read this project and that, in my opinion, it is fully adequate, in scope and quality, as a thesis for the degree of Bachelor of Science in Computer Science.

(Title and Name)

(Project Advisor) _____ _____

This is to confirm that this thesis complies with all the standards set by the Department of Computer Science at the Faculty of Engineering and Architecture at the University of New York Tirana.

Date Seal/Signature

PLAGIARISM CLEARANCE PAGE

I hereby declare that all information in this document has been obtained and presented by academic rules and ethical conduct. I also declare that, as required by these rules and in accordance with the conduct, I have fully cited and referenced all material and results that are not original to this work.

Student First Name, Last name:

Signature:

ABSTRACT

AI RESUME ANALYSER AND JOB MATCHER

Subashi, Andrea.

B.Sc. in Computer Science

Thesis Advisor: Assoc. Prof. Ervin Ramollari

The job search is an inevitable and difficult part of any person's life, be it through online application boards, local recruitment, or strong connections. Today's job market is considered by many to be ruthless and as a result, any additional help to improve the chances of employment should be considered. This application serves as a tool candidates can use to better understand their skills and figure out what possible roles suit them best. Beyond the modern and intuitive UI design, it leverages the power of Artificial Intelligence, Machine Learning, and Natural Language Processing to create a powerful matching algorithm that can find the best job opportunities for a user based on their skills, experience, and education. JobMatcher AI utilizes semantic matching to analyze a user's portfolio and the job postings based on meaning and context, rather than just explicitly typed text like keywords. The gathered information from both sides is then compared producing the final score, which determines how similar what the user offers is to what the position asks for. On top of the core functionalities, the application offers flawless security measures, an informative dashboard, and bookmarking capabilities for any listings of interest. The report details the app's architecture, development process, core functionalities, and possible future additions that can elevate it from an undergraduate project to a truly useful tool for job searching.

Keywords: job, matching, AI, ML, NLP, semantic, search, user (job seeker), keyword, model, algorithm

DEDICATION

I would like to keep the dedication somewhat less formal and personal. First, I want to show my gratitude to the professors whose courses and class interactions are the most memorable to me. Prof Klodjana taught me how to **actually** write in English, how to do research and had some of the most interactive classes as far as I can recall. Prof Esmeralda is the sweetest professor I have had. Prof Nertila's calculus II class, while confusing at times, was probably the most relaxing course I have taken in uni. I have grown fonder of prof Olta the more courses I have had with her, unsurprisingly I have had three courses with her, the most out of any professor. Prof Tahsin was the only one to share food with us in class, not once, but twice, Pistachio Baklava and Cinnamon Rolls, if my memory serves. Not that sharing food with students should become an obligation, but I'm a foodie so those instances were enjoyable. Professor Elton's algorithms and complexity class is the only class that I felt like I was in an IVY League school course. It wasn't just about his knowledge, but mostly how expressive he was and his tone. I don't think I missed one of prof Miralda's classes. Her way of explaining concepts has stuck with me, plus machine learning is cool. Prof Rao was great too. I have one of the most vivid exam memories related to his final exam. The first half felt like I wasn't going to finish any questions and in the second half, one by one I figured out everything. I was actually surprised. The first semester of the third year was the most packed I've ever been, but also the most interesting one overall. Prof Rubin got me interested in communication and networking far more than I expected. It may have to do with how eloquent he is, probably the best English speak in the faculty too. Prof Ervin, I'd say, was the most invested in his lectures. He would, constantly without fail, make the most out of the time in class. He is a great communicator too, partially why I favored him as my supervisor for the graduation project. Prof Klevis made me think the

most as an actual developer and not just a student. His teaching methods were almost entirely based on practice which I'm quite a fan of. Secondly, I'd like to show my gratitude to my high school teachers at Preca College. Not related directly to university but, my three years there definitely prepared me to handle university without much difficulty. I almost exclusively relied on myself for studying, assignments, and projects. However, I would like to thank my friends for the help that they did give me, especially Rei. While in this topic special thanks go to the many people on the internet whose resources I utilized to make any studying available, really. Lastly and most importantly, of course, my deepest appreciation goes to my family for supporting me through these three years, for calling me every day to check up on me and for providing me with anything I ever needed.

TABLE OF CONTENTS

Contents

CHAPTER 1: INTRODUCTION

Background and Context:

The global job market has changed significantly in recent years due to the quick advancements of technology, shifts in required skills for employees, and changes in the workplace. According to the World Economic Forum's Future of Jobs Report of 2025, technology will have the biggest and most varied impact on the evolution of jobs and the workforce soon. 19 million jobs will be created and 9 million will be displaced. AI and processing technology should open 11 million jobs while displacing about 9 million, more than any other trend related to technology (Forum, 2025, pp. 25, 26). These massive changes bring about a complex and confusing environment for job seekers and employers, resulting in what is called the skill gap. The skill gap is: "the discrepancy between the skills a particular role requires, and the skills employees (and potential employees) have." (Coursera, 2024).

Due to the nature of traditional job search platforms, they can fail to correctly capture the connection between the nuanced skill set of a candidate and the listed job requirements. Because of the keyword matching algorithms they use, transferable skills can be overlooked, they can undervalue non-traditional education paths and may not be able to correlate similar experiences that can qualify a candidate for a job they are not considering. From the job seekers' perspective, such a situation can create a problematic situation where they continue applying for jobs not suited to their ability and might miss possible great opportunities.

The introduction of artificial Intelligence, machine learning, and natural language processing offers great solutions to these older challenges. They demonstrate great competence in understanding semantic relations, advanced pattern recognition, and contextual meaning. By

combining these tools there is a significant opportunity to develop intelligent systems that can understand the candidate's capabilities alongside job requirements and perform job matching at far higher rates than traditional means. (Grinblat, 2024)

Problem Statement:

Despite the numerous job-searching platforms available today, an inefficiency in matching candidates with the correct opportunities remains. Many systems rely on keyword searching and a lot of manual filtering by humans which can result in qualified candidates not being considered for a job by the recruiters. This problem creates frustration for candidates, it can increase costs for employers and inherently leads to long stretches of unemployment time for job seekers further hurting their chances of finding a preferable position (Yanamaddi, 2023).

Objectives:

This project aims to develop an AI job-matching website that precisely recommends jobs by analyzing the profiles of the users. The specific objective:

1. Create a system that processes the users' skills, education, and experience using NLP (natural language processing) (Amazon, n.d.) and compares them to the embeddings of the job descriptions.
2. Develop a recommendation system that uses ML (machine learning) (Google, 2025) to select job opportunities that match the user's qualifications the best and rank them based on the matching quality.
3. Create a clean, intuitive, and responsive UI (User Interface) that allows easy account creation, profile editing, and job recommendations.

4. Establish a tight and secure architecture using Next.js (What's in Next.js?, n.d.) as the front-end framework, FastAPI (FastAPI, n.d.) as the backend framework, and Firebase (Firebase, Firebase, n.d.) for the authentication and the database.
5. Evaluate the system through testing and feedback.

Scope:

This project's scope includes the development of a user sign-in and login system using Firebase authentication, the implementation of a user profile that includes skills, education, and work experience field respectively, the development of an intelligent backend to analyze the users' profile, the creation of a recommendation list of suitable jobs, and the design of a responsive front-end UI.

There are several features that are left outside of the scope such as the integration of live online job scrapping, a resume parsing or resume generation functionality, employer-side management tools, detailed analytics or reporting systems, and recommendations on what changes can help land more jobs.

Significance of the study:

This project addresses a point of concern in today's job market by taking advantage of the power of AI to bridge the gap between candidate capabilities and job requirements. The significance of this work can be acknowledged from different perspectives.

For job seekers, it tries to discover possibilities that they may miss otherwise. As a result, it reduces job searching time and unemployment time while increasing chances of finding a role nicely suited for their skillset.

From a technical point of view, this project uses new and powerful tools like AI, ML, and NLP in practical ways in a real-life scenario. It also serves as a contribution to the betterment of intelligent systems that can interpret human skills and experience.

From a social perspective, the project can help with better usage of human resources, a chance of high job satisfaction, and lower unemployment time periods.

Methodology Overview:

The first step was to gather as much information as possible related to the domain of the project. This included similar software, publishments regarding the job market and its tendencies and possible gaps that could be filled.

The tech stack also holds importance. I needed to find the overall best suited technologies for the task. After doing research I decided to use Firebase for its enhanced security, FastAPI because of its high performance and Next.js for its support and ease of deployment. NLP was used to extract the meaning of the user's profile, and the job descriptions while ML was used to create the recommendation system.

An AGILE approach was adopted (Atlassian, n.d.) to jump straight into development, while testing and refactoring along the way. This approach ensures continuous progression of work and consistent testing for any roadblock along the way.

The site's performance was measured through the relevance of recommendations, speed of response, and user satisfaction.

Structure of the Report:

The remaining part of the report is organized in the following manner:

Chapter 2: Literature review shows existing research and systems for job matching, the AI used in recruitment, and machine learning and NLP techniques that are used to inform this project.

Chapter 3: Requirements Analysis and System Design details the functional and non-functional requirements, the higher-level architecture, a detailed design using UML (unified modeling language) and ERD (entity relationship diagram), the constraints, the UI design, the technologies selected in detail, and the security measures.

Chapter 4: Implementation, Testing, and Evaluation describe the description of the development environment, the practices and standards of the project, illustration of key features implementation, UI implementation, the system configuration, strategies used for testing, test cases, noticeable bugs, a comprehensive performance evaluation, and feedback from the users.

Chapter 5: Recommendations and conclusion include a summary of the project achievements, learning outcomes, a thorough evaluation of the objectives, what the main challenges of development include, recommendations for future work, what the impact of the project might be in the future, and the conclusion.

Transitional section: Having gone over the structure of the project, the next chapter will talk about the existing literature, the tools, and technologies related to said tools that contextualize the job searching scene.

CHAPTER 2: REVIEW OF LITERATURE/BACKGROUND STUDY

Overview of the Domain:

A fundamental challenge in the job market is matching individuals to suitable job openings often referred to as talent acquisition. The OECD's Program for the International Assessment of Adult Competencies (PIAAC) started a survey in 2011 regarding the proficiency of adult workers. The survey was called the Survey of Adult Skills, and its main goal was to provide data on workers, skills. The uniqueness of this survey was that the skill level was assessed and not self-declared. Up until 2019 over 250000 individuals in 39 countries were assessed. The findings for the OCED researchers show that about 35% of workers were mismatched by their quality of work, where 22% were overqualified and 12% were underqualified. While the mismatching is not caused by the skill gap exclusively, it is labeled as one of the main reasons in the research (Roland Rathelot, 2023). The large amount of applications and positions in the modern market has rendered purely manual methods of matching candidates to proper roles inefficient in terms of financial cost and time.

As a solution to the problem, ATS (Applicant tracking system) was created. It is a digital tool that helps by simplifying administrative tasks and enabling applicant filtering. The central issue of ATS was the heavy reliance on keyword matching which proved ineffective enough to require the development of more advanced systems. As mentioned by the International Research Journal of Engineering and Technology (IRJET), ATS has several problems including an algorithmic bias that can undervalue certain demographic groups. One such case is Amazon's bias issue pointing out the overreliance of the system on historical data potentially causing biases (Samarth Gore, 2025).

To combat the limitations of traditional ATS AI and NLP have become increasingly essential to the modern market. They focus on semantic relationships, meaning: the connection of words based on the meaning of user profiles and job requirements. The main tools used in this domain are sophisticated matching algorithms, automated extraction of data such as skills or experience, and semantic matching. Semantic matching, for instance, is a technique to determine if two elements (words, phrases, sentences, paragraphs, etc....) have a similar meaning. Often used with text embeddings, it turns unstructured data (emails, images, videos) into structured data (Excel files, SQL (structured query language) databases, search engine optimization (SEO) tags) and connects applicants to the correct jobs even if the keywords do not match entirely (Ken Gu, 2021).

The current landscape of HR (Human Resources) tends toward the adaptation of AI solutions aimed at improving speed, quality, and costs. It was stated that companies that adopted AI in recruiting cut costs by 30% in that department and time to hire was cut down by 50%. While not all metrics support the usage of AI due to some cases of high variance and standard deviation overall, it is an efficient and useful tool for the recruiting procedure (Ouakili, 2025).

Review of Existing Applications:

The hiring process has dramatically changed after the introduction of digital systems, evolving from simple job listings to powerful AI-driven software. This section of the report highlights the existing applications in the domain focusing on their features, strengths, and limitations, and identifies areas of improvement that this project will try to tackle.

Job Boards (LinkedIn, Indeed, Glassdoor)

These are the biggest companies and dominate the recruitment market. Their strength relies on the vast databases of job listings they possess, and their reach providing recruiters with a large pool of candidates and applicants with many job positions. They integrate basic AI for recommendations based on user skills, titles, work experience, interests, and search history, all extracted from user activity or provided by them. However, a major limitation of these systems is the opaque nature of their algorithms. Users do not receive any specific feedback as to why certain roles are recommended. While these companies are implementing semantic search more in their systems the depth of understanding, specifically concerned with nuanced requirements, can vary and may favor explicit word searching instead. This can lead to a lot of suggestions with questionable relevance, combined with minimal feedback on how the user can improve their profile for better matching.

Specialized AI-driven profile analyzers

A rising trend includes specialized AI tools that focus on deep profile analysis and job matching such as JobScan (Jobscan, 2025) and VMock (vmock, n.d.). JobScan focuses on optimizing resumes to pass the ATS test. It even claims to increase interview chances by 50% (Jobscan, 2025). VMock optimizes the resume for finding curated job listings. In general, their strength lies in providing detailed feedback and more specific job matching. However, due to the detail, these searches result in a narrower scope compared to major boards. On top of that AI techniques, while innovative, can have varying results and high standard deviation. Advanced features are also almost always paywalled.

ATS and AI features

ATS is a software that helps companies simplify the hiring process by collecting resumes submitted online, ranking applicants based on keyword matching and qualifications (not all), filtering out resumes that do not satisfy the system requirements, and tracking the candidate's progress through the hiring process. They are widespread. JobScan research shows that over 98% of all Fortune 500 companies use ATS. The most relevant from the findings are Greenhouse, Lever, Workday, and iCIMS. These systems are relevant because they represent the first part of the recruitment process, however, their employer's centric nature does not favor the applicants (Henderson, 2025).

Identified Gaps

Several gaps emerge while analyzing these applications. The primary concern is the lack of transparency concerning matching algorithms and feedback. Often users are left to infer as to why their resumes fail an ATS scan or why certain matches are presented to them. While semantic search is advancing rapidly the ability of large-scale models to fully digest, analyze, and find the best possible outcomes beyond keyword matching appears to have room for improvement.

This project aims to address these issues by focusing on a transparent and robust matching methodology that incorporates robust NLP techniques: hybrid scoring combining semantic embeddings of comprehensive user profiles with explicit skill matching.

Technologies and Tools:

Programming Languages & Core Frameworks: Python is the dominant programming language in this domain due to its rich selection of libraries for data science, machine learning, and NLP. Backend frameworks such as FastAPI which excels at machine learning and NLP technologies while offering high performance and scalability, Django as a full-featured web framework, and Flask, a lightweight framework that excels in creating APIs (Application programming Interface) due to its simplicity and ease of use. For the front end, JavaScript and TypeScript are predominant, alongside popular frameworks such as React, usually combined with Next.js for static site generation, server-side rendering, and optimized routing; Angular, and Vue.js. serve as alternative front-end frameworks while Node.js using frameworks like express.js offer an alternative for the backend.

Databases & Data Storage: Database selection corresponds with the type and quantity of data. Relational databases like MySQL and PostgreSQL are used for storing well-structured data like user profile information, and job postings with well-defined schemas. NoSQL has become more popular due to its flexibility. One group includes document databases like MongoDB and Firebase Firestore (utilized in a lot of web applications due to its real-time capabilities and ease of integration). They are suited for handling semi-structured data, data that doesn't follow the tabular structure associated with relational databases, or other forms of data tables (What is Semi-Structured Data?, n.d.). Applications that require full-text search and keyword search engines like Elasticsearch or OpenSearch are often integrated. Recently vector databases like Pinecone have been used due to their ability to store and query high-dimensional embeddings generated by NLP models, an important part of semantic search.

AI, NLP, and Machine Learning Libraries: This part is a core component of today's job matching systems and Python leads with its powerful libraries.

Base NLP Processing: SpaCy is widely used for tokenization, part of speech tagging (assigning each word in a document a part of speech), and named entity recognition (categorizes and extracts most important pieces of information from unstructured text). NLTK or the natural language toolkit is great, especially for education and research. (spaCy, 2025)

ML: Scikit-learn is a library that provides many algorithms for task classification. TF-IDF vectorization (a technique used to convert text into numerical vectors) and clustering. TensorFlow and PyTorch (PyTorch documentation, n.d.) are the leading frameworks for deep learning.

Sentence embeddings& transformers: The sentence-transformers library has simplified state-of-the-art models like BERT for generating vector representations of text. They are essential for semantic similarity calculations. The Hugging Face Transformers library provides more in-depth models and tools for NLP work (Face, Sentence Transformers, 2025).

Cloud Platforms & Deployment Tools: Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure provide scalable infrastructure for hosting apps, databases, and deploying ML models. Containerization software like Docker powered by Kubernetes is today’s standard for deploying environments and managing an application’s scalability. Git is used for version control while GitHub and GitLab are essential today for collaborative development and code management.

This project uses FastAPI as the backend framework, Next.js for the front end, firebase for both user authentication and the database sentence-transformer library models regarding the NLP and ML portions.

Methodological Approaches:

Software Development Methodologies: Agile methodologies are widely used in making software applications including AI-powered web apps. They allow features and model performance to be reassessed consistently in development, which is crucial when dealing with ML where outcomes cannot be predicted with absolute accuracy and experimentation plays a vital role. Sprints (set periods of time assigned for the completion of a specific task) can be used to split the model training, testing, and integration steps into cycles, with user feedback influencing the sequential cycles (Rehkopf, n.d.). This method differs significantly from the Waterfall method which adopts a linear and rigid progression type. It is not recommended for AI development due to the lack of concern for its volatility and exploratory nature.

DevOps is an important part of development extending beyond traditional software with MLOps (machine learning operation). DevOps puts emphasis on continuous Integration/Continuous Deployment (CI/CD), automation, and the collaboration of different teams to ensure the ML models and the applications using them can be developed, tested, deployed, and maintained efficiently (Center, n.d.). This part is significantly important for the performance of the matching algorithms over time.

Key Software Engineering Principles & Practices: Modular design: An application that implements a loosely coupled method (components have minimal dependencies on each other)

was implemented in this project. This separation helps with developing and testing certain parts of the app independently. One such case is that the matching algorithm can be updated without requiring the redeployment of the front-end UI.

API-First Design: Since the project has a clear separation between the front end and the back end, defining a clear API contract early assures parallel development and properly defined interactions between the UI and the matching logic.

Version Control: Git is essential today for storing and versioning code, collaborative application development, and providing a layer of security and insurance. It allows reverting to previous versions of code if need be and makes it easy to trace potential problems.

Comprehensive testing: Ai-powered applications require extensive testing. This means validating the model's performance compared to predefined metrics of measurements, testing for robustness by using edge cases like unique or uncommon inputs and monitoring concept drift (change in the input-output relationships an ML model has learned) that can cause the degradation of model accuracy.

Iterative Model Development and Evaluation: In many cases, applications will use pre-trained ML models like sentence transformers. Even in such cases, these models need to be fine-tuned for the task they are being utilized for, in this context of job matching. This is done to guide improvement and ultimately achieve high performance.

Theoretical Frameworks:

Information Retrieval (IR) Models: At a baseline, the process of matching jobs to correct user profiles is an information retrieval task. The foundational IR model in this case is the Vector Space Model (VSM). Using VSM the contents of test documents are represented as vectors in high dimensional space where each feature corresponds to a dimension. The similarity between such documents is usually calculated using Cosine Similarity by finding the cosine angle between vectors. A cosine value close to 1 or a smaller angle show a lot of similarity (Singh, 2022). This project's core principle is the usage of VSM techniques by converting profile text and job descriptions into embeddings and applying cosine similarity to determine their semantic correlation. These embeddings are derived from neural networks.

Natural Language Processing (NLP) Theories & Models: Distributional Semantics which says words occurring in similar contexts often share the same meaning forms the root of modern sentence embeddings. This concept has been applied with great success through neural networks, especially transformer models. Such architectures include BERT, RoBERTa, MPNet, and the more lightweight versions like MiniML. These models weigh the significance of words in a sequence and generate contextual embeddings. These projects utilize the sentence-transforms library, which contains a wide array of pre-trained Transformer models. Models like all-MiniLM-L6-v2 and all-mpnet-base-v2 are utilized to map user profiles and job descriptions to multidimensional dense vector spaces (Face, all-MiniLM-L6-v2, 2025) (Face, all-mpnet-base-v2, 2025). This way the matching process is significantly improved compared to simple keyword overlapping by capturing underlying relationships in the text.

Machine Learning for Similarity and Ranking: While the models used in this project are pre-trained, they are fundamentally rooted in ML techniques. Sentence-transformers are results

of large-scale supervised and unsupervised learning on massive text corpora. The main ML task of this project is similarity learning, specifically calculating the cosine similarity between the vectors created for the user profile and the vectors made for the job listings. A score is given which is used to rank jobs for the user. This is a technique used in learning-to-rank and information retrieval systems. Pre-trained models are widely adopted ML approaches for "zero-shot" (prompting an LLM (large language model) with no examples to assert its reasoning capabilities) and "few-shot" (prompting an LLM with few concrete examples) matching (GeeksForGeeks, 2024). Future additions may incorporate ideas from Recommender Systems theory like content-based (uses item features to return relevant items) filtering and collaborative filtering (recommendations based on other users with similar behavior and preferences).

Critical Analysis of Existing Solutions:

A primary problem, present in many job board applications like LinkedIn Indeed, and ATS, is the lack of transparency in the matching algorithms they use. Both applicants and employers receive matches and rankings with little insight into the factors giving these results. Such approaches are referred to as black box approaches, meaning that the inner workings of the systems are not public. This leads to a lack of trust from the user since the relevance of the matching algorithms is impossible to assess firsthand.

Secondly, while semantic search is advancing rapidly, many systems still rely heavily on keyword-based matching and simple semantic analysis. While this method can give satisfactory results it can miss holistic data about the user causing specific profiles not to be considered for jobs, they are otherwise capable of performing.

One other area for improvement is the utilization of the user profile. In many scenarios, systems will rely on resume parsing, a process that leads to data loss. Even in scenarios where the profile input is provided by the system, it may not incorporate education and work experience in its algorithms defaulting to skill tags and job titles. This project uses skills, education, and experience to generate rich semantic embeddings that represent the candidate thoroughly.

Lastly, the feedback for users is often shallow and unconstructive. It lacks the information to significantly alter the user profile to pass ATS or match with desired positions. This method focuses on a more transparent approach and aims to implement valuable feedback in the future.

Overall, the main gaps are the lack of transparency in the matching process, the overreliance on keywords, and the underutilization of advanced ML techniques. This project tries to address the above-mentioned issues by using a hybrid scoring model that combines deep semantic embeddings taken from user profiles (including skills and experience) with explicit keyword analysis to connect talent with employers.

Summary of Opportunities:

The main opportunity is the development of systems that use more advanced semantic understanding and transparent matching procedures. There is a need for apps that can interpret detailed user profiles including their education and particularly job experience, beyond simple skill tagging.

Transitional section: With a good understanding of the domain and the currently employed techniques the project can proceed to define the system's requirements and design. This part makes sure the solution to the problem is correct and acceptable.

CHAPTER 3: REQUIREMENTS ANALYSIS AND SYSTEM DESIGN

Identification of Stakeholders:

Job Seekers

Interest: Job seekers are interested in using the application because it offers a powerful job-matching tool that implements AI and machine learning to best fit the profile of said user.

Expectation: Efficient and precise job discovery, an intuitive profile management interface, and control over their data.

The developer

Interest: Create an appropriate project conforming to the rules and guidelines of the graduation project schema.

Expectations: Access to tools, technologies, and support throughout the development process and the creation of a resume-worthy project.

Project supervisor

Interest: To ensure the project meets all the requirements set by the school, the finalization of both the software project and its corresponding report, the demonstration of sufficient software engineering and problem-solving skills by the students, the effective application of AI and ML techniques.

Expectations: The completion of the project and the report, meeting the deadline, transparency from the developer, and a complete understating of the project features and functionality by the developer.

Gathering of Requirements:

Research done by the developer

Method: Thorough research on job matching applications, resume analysis tools, the online recruitment process, the application of AI and ML in the domain, and the challenges faced by recruiters and job seekers. On the technical front: choosing the right technologies for the task like the tech stack and the ml models

Outcome: A decent understanding of the domain, existing technologies and deployed systems, limitations, and opportunities for improvement using AI/ML/NLP to enhance the matching process.

Proposal of project and supervisor feedback

Method: Several project topics were discussed, and this one was decided on because it is a relevant topic of today, and an AI system implementation is a good addition to the project. The feedback from the supervisor helped refine the scope of the project and explain the academic requirements. Weekly meetings were conducted to maintain a good pace of work while clarifying uncertainties.

Outcome: Updated scope for the project to a realistic goal confined by several limitations like time and experience, practical implementation of features like the change from resume parsing to a user profile for displaying the skills, experience of work, and education.

Functional Requirements:

1. User Authentication
    1.1. User Registration
    1.2. User Login
    1.3. User Logout
2. Profile management
    2.1. View profile
    2.2. Manage skills
    2.3. Manage job experience
    2.4. Manage education
3. Job matching
    3.1. Profile extraction for matching
    3.2. Job listing data for matching
    3.3. Semantic profile embedding generation
    3.4. Job description embedding generation
    3.5. Semantic similarity calculations
    3.6. Keyword similarity calculations
    3.7. Hybrid score calculations ➔ using semantic and keyword similarity scores for a balanced final score
    3.8. Display job matches ➔ most relevant ones
    3.9. Filter and Rank job matches ➔ based on the hybrid score system
    3.10. Bookmark job listings
4. UX & UI

4.1. Landing page ➔ create an account or log into your existing account

4.2. Navbar ➔ navigate through the pages

4.3. Dashboard ➔ confirms the login and displays account credentials

Non-Functional Requirements:

Performance: The backend APIs for profile management respond within a second from development testing. Frontend Page load time: After running the server the first time the page loads can take several seconds. This is due to outdated hardware and the necessity to restart the server because of development, while deployed this should be no issue. Backend Load time: The server takes several seconds to load, however that number was cut down significantly by using caching to store the job listings embeddings locally. It suffers from the same issues as the front-end page.

Usability: The user should be able to understand how to use the site immediately, it is simple, the profile setup takes a short time, and the matching process is done automatically after the profile is updated. Feedback: The system provides clear and timely feedback to users for successfully updating their profile, loading times, and possible errors. Accessibility: The application adheres to basic website usability guidelines. Reliability: The project aims for high availability. Integrity: The data stored from the user in the database is accurately received.

Scalability: The project was developed with a limited scale in mind, so the design was not inherently focused on creating a scalable system. A drawback is that all the job listings are loaded in memory and with very large datasets this can significantly lower speed. Security:

Authentication is handled by Firebase Authentication and the database uses Firebase Firestore, both very secure. User data is protected by Firestore security rules. User identity is verified using Firebase ID tokens in the backend.

Portability: The frontend app is compatible with all stable versions of popular browsers.

Constraints and Assumptions:

- Timeframe: The project and the report are to be completed in about 12-13 weeks. This timeframe does not allow for very advanced development and prioritizes a focused scope and core features.
- Group division: The project is to be completed by a single student, significantly lowering the number of features and parallel programming that can be done.
- Data source: The job listings are taken from the Kaggle (Kaggle, n.d.) dataset. The application does escape job listings online or use real-time APIs due to complexity, features gated behind paywalls, and time.
- Computational Resources: The usage of powerful ML models, and dataset size are limited by local hardware. Large-scale services are out of scope.
- Budget: The project relies on not having a budget; thus, all technologies rely on free-tier services
- Dataset: The chosen dataset is assumed to be diverse and formatted well enough so the AI matching algorithm can take full advantage of it. It does not represent the real job market.

- Pre-trained model efficiency: It is assumed that the model is powerful enough to achieve meaningful results without having to do custom model training or extensive fine-tuning.
- Language: The application language is English and the NLP models used are English-focused.
- Security: It is assumed that the security that the frameworks for the frontend and backend in combination with Firebase Authentication is sufficient for the scope of this project.

High-Level Architecture:

Frontend

Technology: Next.js with TypeScript and Tailwind CSS (CSS, n.d.)

Responsibilities include the UI / UX, handling user interaction such as profile creation/ management and initiate matching requests, client-side state management, communication with Backend API via HTTP requests, and interaction with Firebase Auth for sign-up, login, and session management for client

Backend

Technologies: FastAPI

Responsibilities include providing RESTful (architecture style that defines constraints of how the architecture of a system should behave) (Fielding, 2000) APIs for the frontend, handling logic for user profiles and matching, verifying user authentication by validating ID tokens, interacting with Firebase for CRUD (Create, Read, Update, Delete) operations on user data, managing the job dataset, loading and using the sentence transformers model, generating

semantic embeddings for user profiles and job descriptions, implementing a hybrid scoring and ranking system, and managing caching for job embeddings.

Database and AUTH

Technology: Firebase Authentication and Firestore

Responsibilities: Secure user auth and storage of data.

Dataset and ML Model

Dataset: Kaggle CSV dataset

ML model: all-mpnet-base-v2

Responsibilities: The data set provides the data corpus used by the model to convert it to semantic vector embeddings.

Justification for architecture

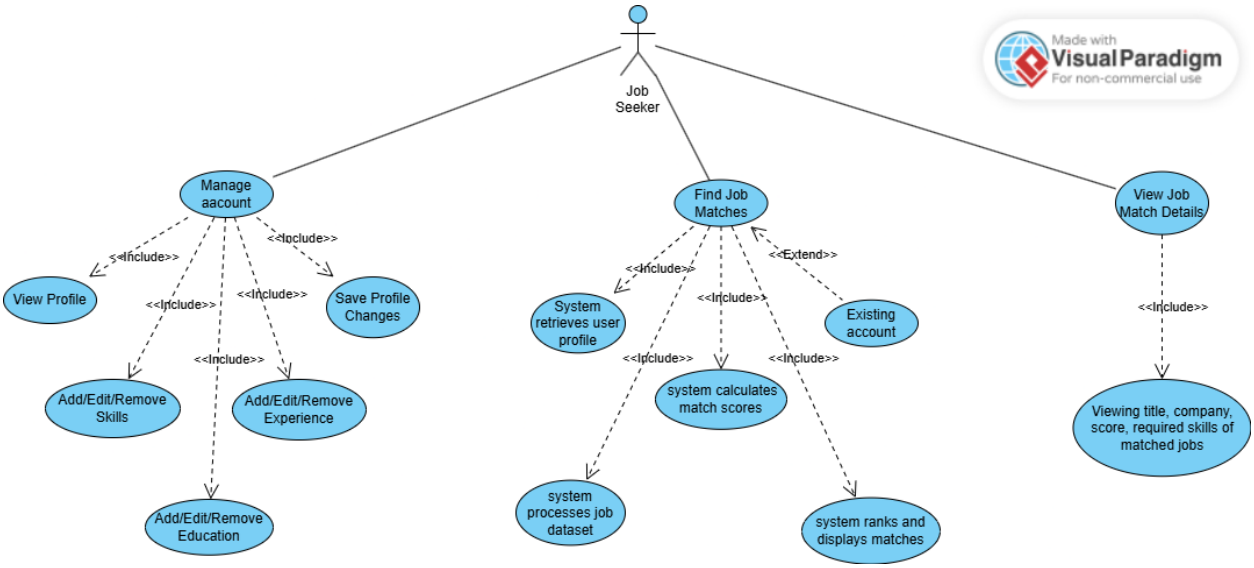
Separation of work: Client-server model to separate the frontend from the backend logic. Improves modularity, thus speeding up the development process.

Technology specialization: It allows the developer to choose the best technology for each task.

Alignment to scope: Robust enough to develop all core features for the project without introducing unnecessary complexity.

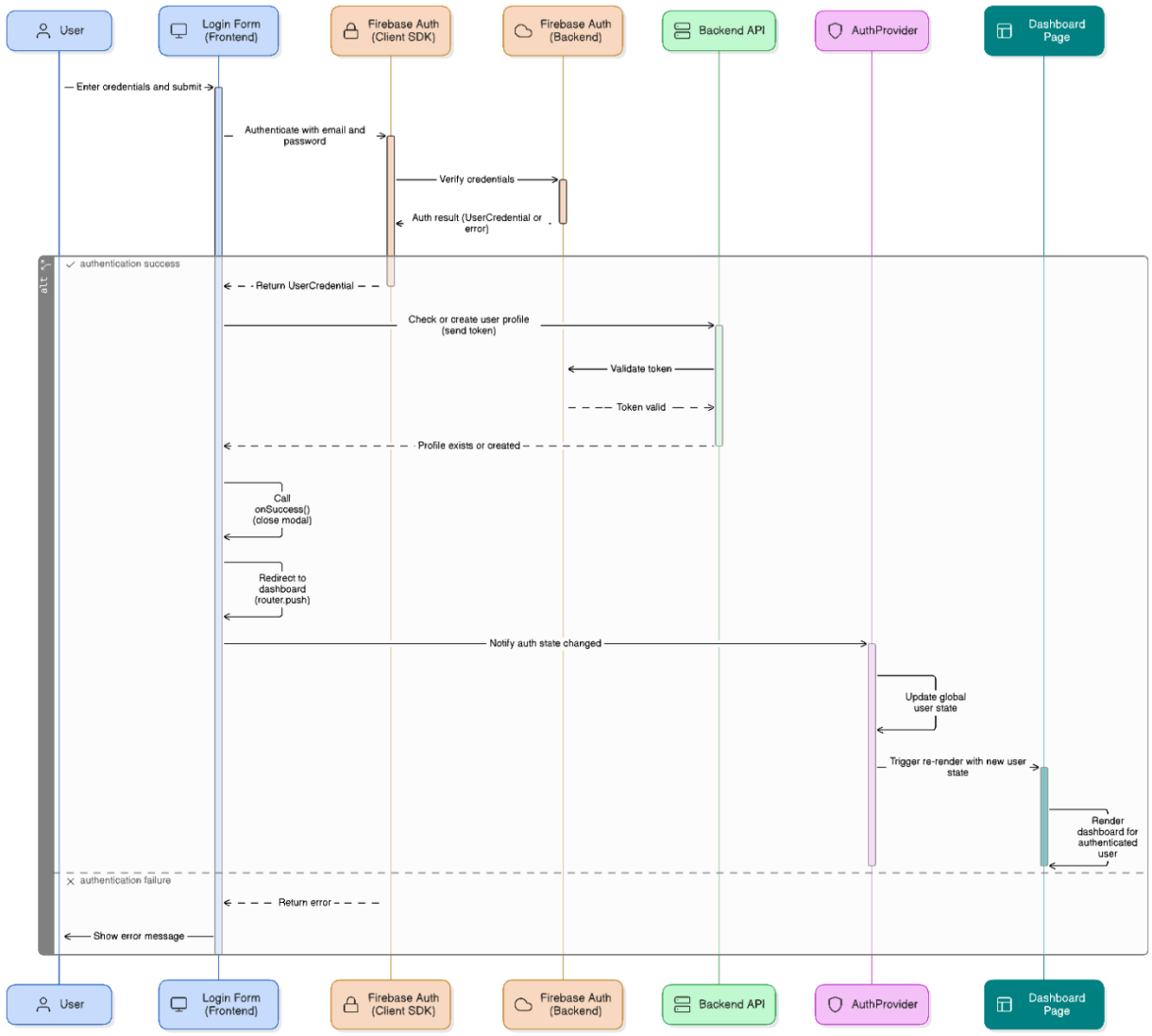
Detailed Design:

UML Diagram



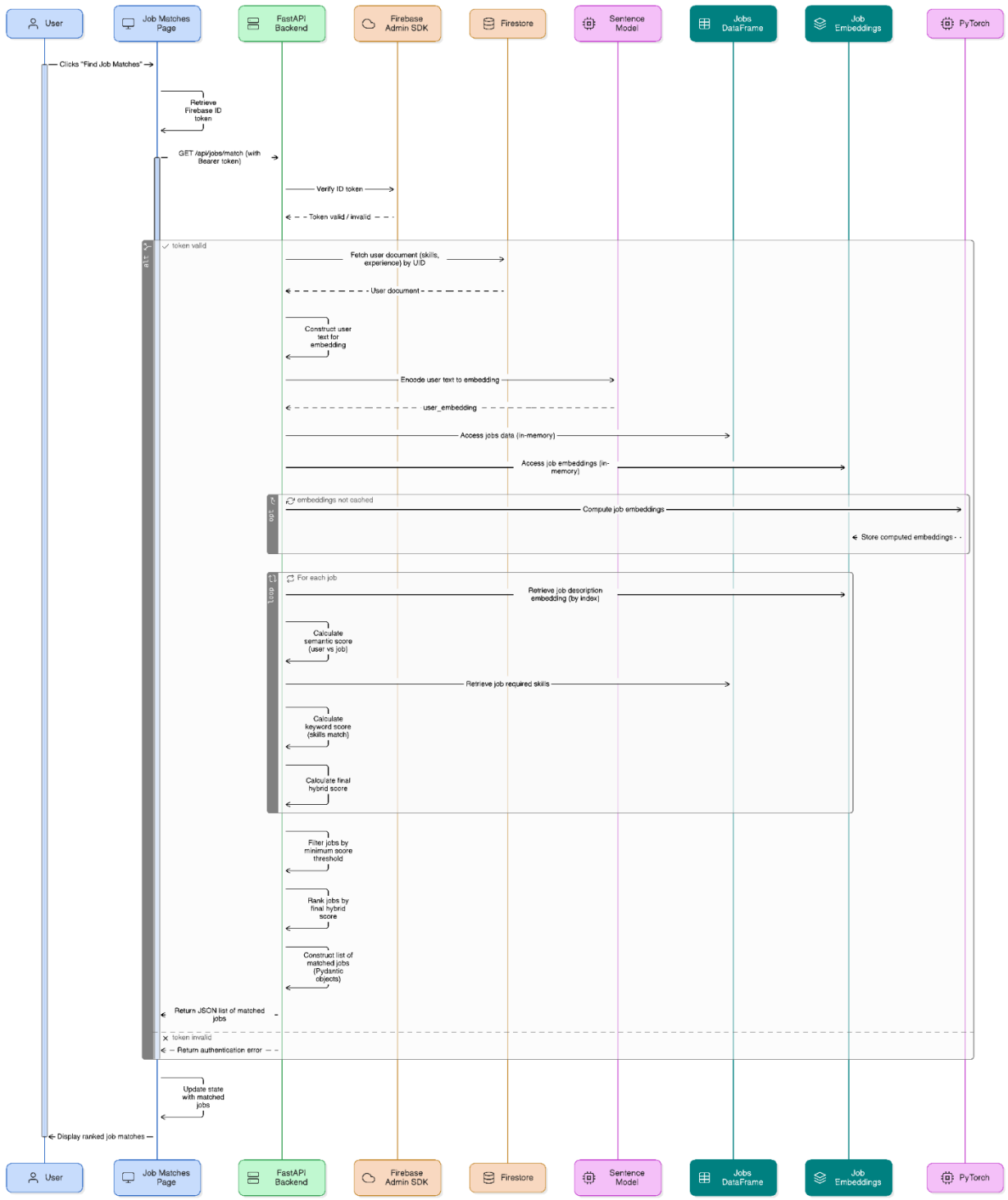
The UML diagram gives an overview of how the user interacts with the application.

Sequence Diagram: User Login



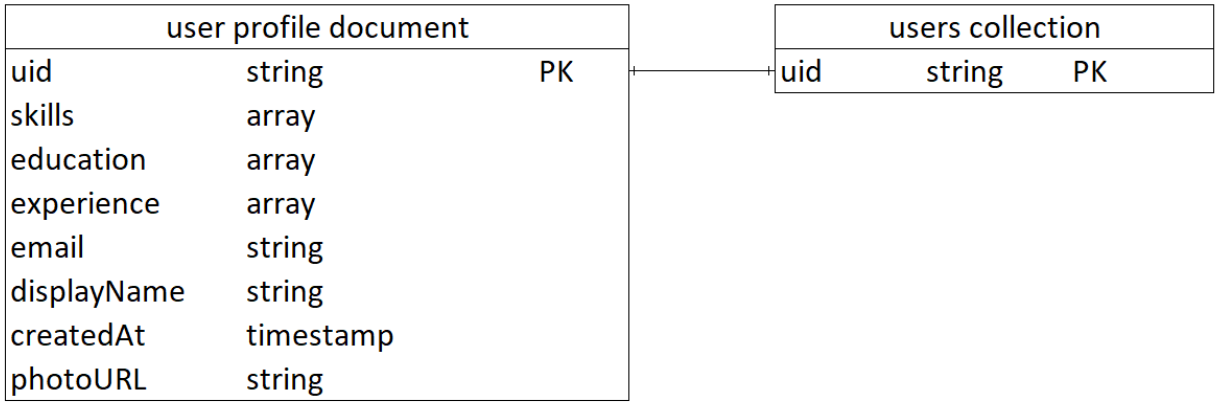
This sequence diagram shows the authentication part of the application usage flow in detail.

Sequence Diagram: Job Matching Request



This sequence diagram goes over the matching part of the application usage flow.

Entity-relationship diagram for database design



The ERD diagram displays a one-to-one connection between the user profile document and the users' collection. Firebase uses collections as objects and each of these entities has its document/s. In this case, one document is assigned per user.

User Interface Design:

UX considerations

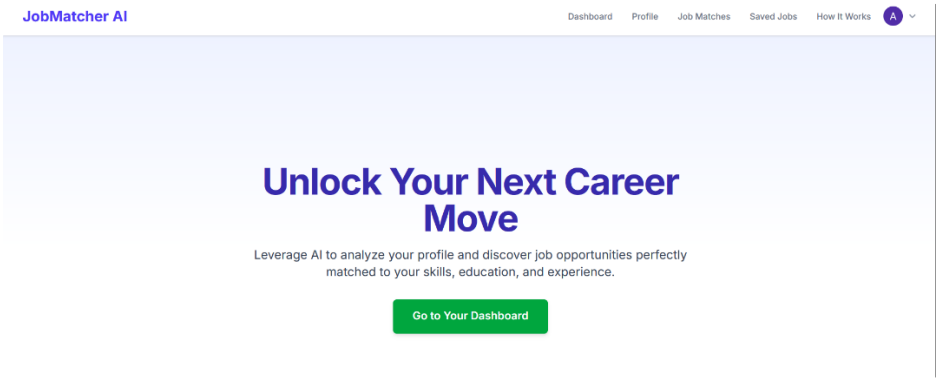
Such considerations include the efficiency to complete core tasks quickly, visual feedback to user actions, the Consistency to use the same layout patterns in all the pages, and error prevention and recovery to minimize input errors and provide helpful error messages.

Accessibility features

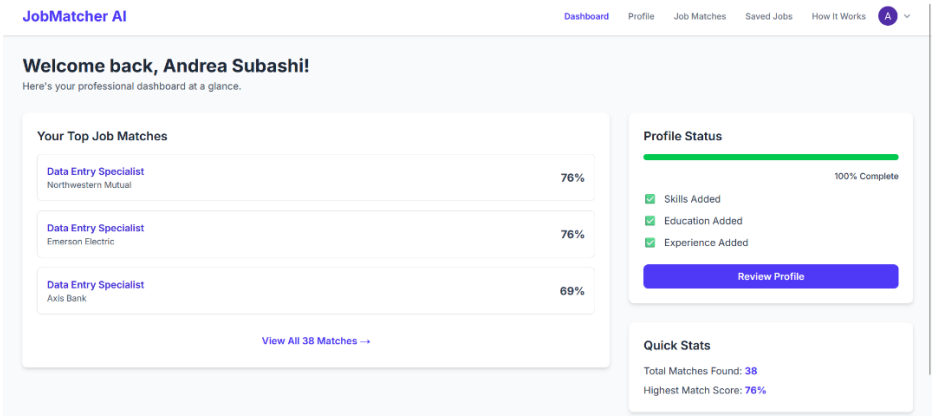
Semantic HTML, color contrast based on the WCAG AA guidelines, managed through the Tailwind color choices, visibility so all elements are easily distinguishable from one another

Demo of the application

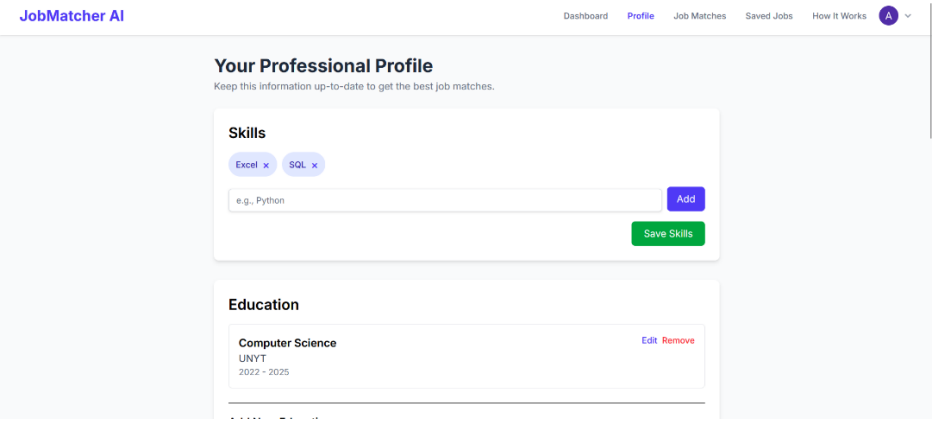
Landing Page



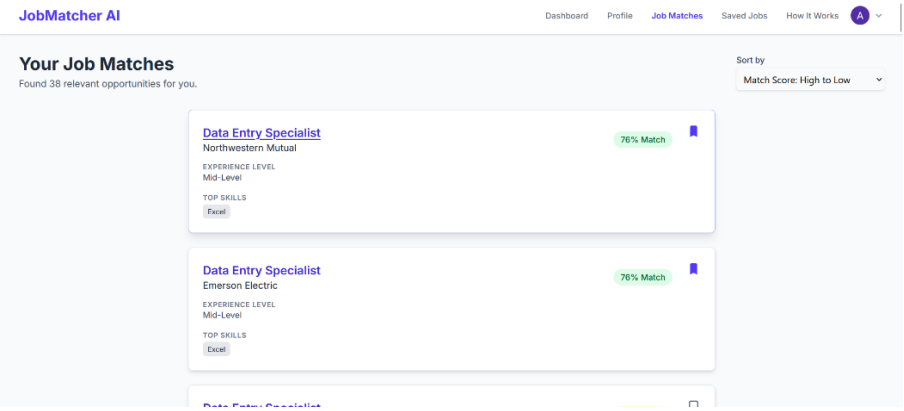
Dashboard



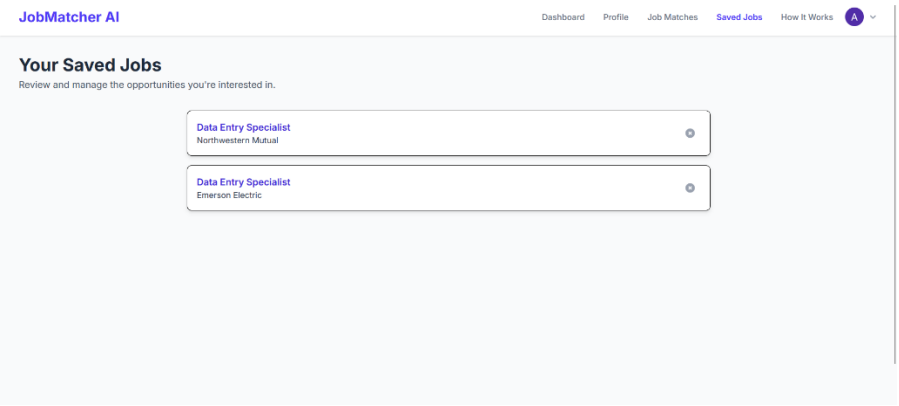
Profile



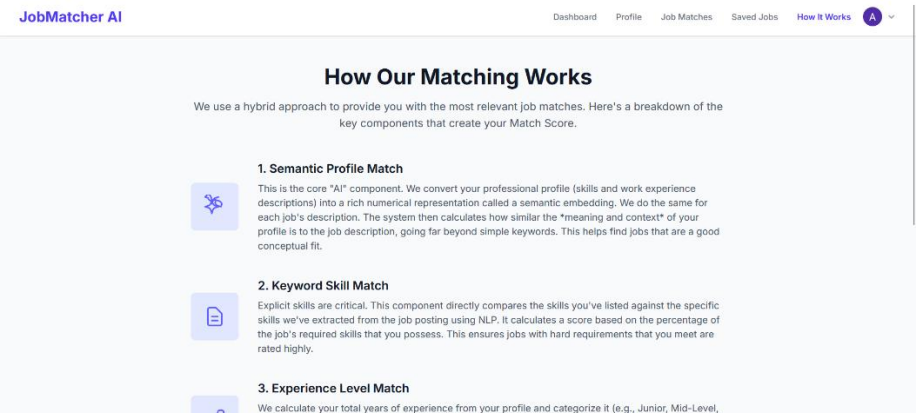
Job matches



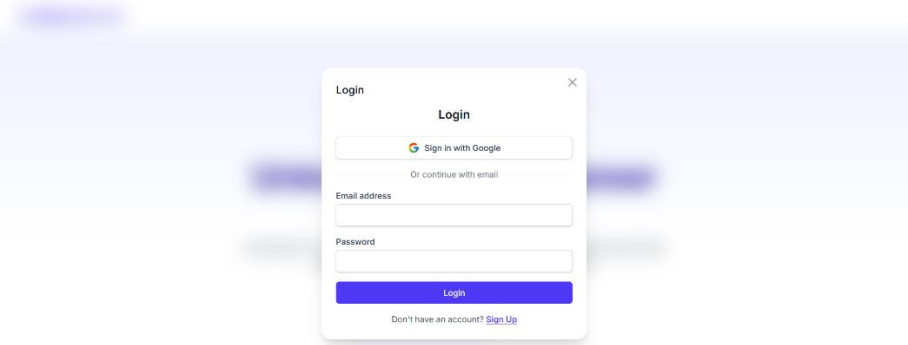
Saved Jobs



How it works



Login/Signup



- Watch showcase video

Technology Selection:

Frontend

Nexjt.js was Selected because of its server-side rendering and static site generation capabilities, optimized routing, and good developer experience. It is a component-based architecture that is modular and reusable. React is the underlying library for next.js, selected because of its popularity, support, and developer tools. TypeScript was chosen over JavaScript because of its static typing and the ability to write clearer, less confusing code and to catch errors early. Tailwind CSS is a practical CSS framework that focuses on a direct approach to styling sites. It tries to avoid writing extensive custom CSS by providing many ready-to-use designs. This project's focus is functionality, so Tailwind's approach suits the scope. Headless UI is used to create unstilled UI components that incorporate greatly with Tailwind.

Backend

Python was chosen because it is the most supported language for machine learning due to the many libraries and frameworks designed for it. It also is a language I am familiar with which helps with development speed. FastAPI was selected because it is a fast web framework for Python. Because Python is a slower language a framework that actively improves it is beneficial.

Libraries:

os – used to interact with the operating system

ast – parses strings with Python expressions into syntax trees

uuid – manages universally unique identifiers

datetime & date – manages date and time

re - provides expression matching operations (Python, re — Regular expression operations, n.d.)

pandas – data analysis and manipulation

fastapi – library for the backend

fastapi.middleware.cors – enables frontend to communicate with backend

pydantic – validates data via Python classes (Pydantic, n.d.)

typing - provides a vocabulary of more advanced type hints (Python, typing — Support for type hints, n.d.)

dotenv – loads environmental variables from a .env file

dateutil.parser – parses dates into Python objects

dateutil.relativedelta – more convenient date calculations than timedelta

Uvicorn: server used to run FastAPI app.

Database and authentication

Firebase Authentication: Selected because it is easy and fast to implement, it is secure and provides many authentication possibilities like Google sign-in and email/password. It manages user sessions and integrates well with Next.js.

Firestore: IT has a flexible schema that is suitable for user profiles with varying structures for their skills, education, and experience. It has a great free tire.

AI/NLP

Sentence-transformers Library was chosen for its ease of implementation and the powerful sentence and text embedding generation. It also includes many pre-trained models that have been exclusively trained for semantic similarity tasks.

Pretrained models: Selected to provide state-of-the-art NLP capabilities without having to train the model independently which is out of the scope of the timeframe

PyTorch: an underlying framework for sentence-transformers

Version control

Git & GitHub: Provide version control and code management

Security Design:

Authentication

Mechanics: Managed by Firebase Authentication. Provides industry-standard features to handle signup, login, password hashing, and session management.

Mitigation: Authentication is a sensitive part of any project and offloading it to a reputable service such as Firebase lowers the risk of vulnerabilities with self-built systems.

Authorization

API endpoint protection: All backend endpoints are protected are protected by requiring a valid Firebase ID Token.

Token verification: get_current_user dependency in FastAPI verifies the received ID token using Firebase Admin SDK (Firebase, Add the Firebase Admin SDK to your server, n.d.). It checks the token's signature, expiration, and if it has been revoked to ensure the user is legitimate.

Data access control: When modifying data in Firestore the backend uses the user_uid obtained from the verified token to ensure the modifications are being done by the correct user.

Data Encryption

Transmission: All communication happens through HTTPS in a production-like environment. It encrypts data exchange making it impossible for man-in-the-middle attacks or eavesdropping.

Input validation & Sanitizations

Backend: The Pydantic model's requests are automatically validated which helps prevent injection attacks and ensures the integrity of data before being manipulated.

Frontend: Next.js and React protect against cross-site scripting vulnerabilities by default.

Secret management: Sensitive information like the Firebase Admin SDK service account key is saved in the backend directory and included in the .gitignore file.

Summary

The client-server architecture using Next.js FastAPI and Firebabe is modular and takes advantage of the strengths of each of the technologies implemented. The detailed design section explains the module interactions, the data structure used through Pydantic models, the API endpoints, and the Firestore schema. The UI principles prioritize simplicity and ease of use while remaining modern and professional. Lastly, security measures were stated. The combination of these elements provides a solid framework for the app's development.

Transitional section: After stating the system design and its requirements, the next chapter focuses on the implementation process which discusses the important parts of the development, the application evaluation metrics and how they were validated.

CHAPTER 4: IMPLEMENTATION, TESTING, AND EVALUATION

Development Environment:

Operating system: Windows 11

IDE: VS Code because it has a large community, is lightweight, has access to a multitude of extensions, has an integrated terminal, and has seamless git support.

Programming languages & Runtime

Python version 3.12.3(the latest version as of the start of development): because of the libraries for ML, NLP, data manipulation, and familiarity.

Node.js: because it is required as a runtime for next.js and provides the npm packet manager (manages dependencies)

TypeScript: because it adds static typing and improves code quality and development speed by catching errors during development.

Web browsers: Chrome, Brave, Mozilla Firefox, Edge: to test the front end of the application.

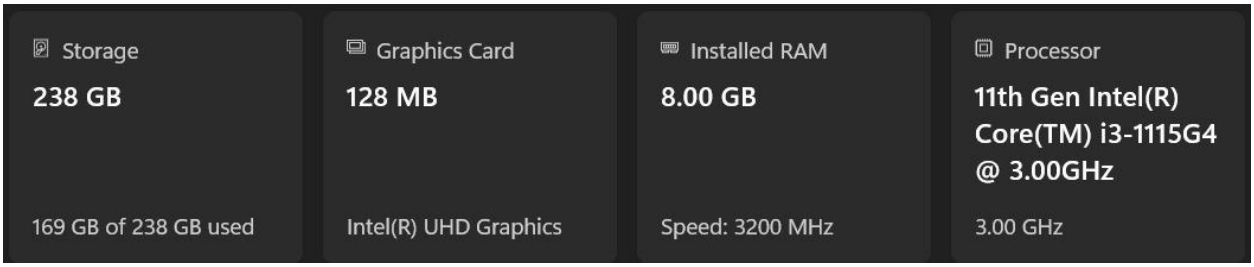
API Tool: Postman because it allows the sending of HTTP requests with custom headers and bodies.

Virtual Environment Manager

venv: because it is Python's built-in module for creating virtual environments and creates project-exclusive dependencies.

Packet manager: pip: for python npm: for Node

Hardware



Coding Practices and Standards:

Conventions

The backend followed PEP 8(Python Enhancement Proposals 8) for the format and the naming conventions (snake_case for functions and variables and CapWords for classes). The front end follows standard JS/TS conventions. Prettier (VS Code extension) was used to enforce consistent code styling and formatting. camelCase was used for functions and variables and PascalCase was used for React components.

Modularity and Reusability

In the frontend React components like Modal.tsx, LoginForm.tsx, and Navbar.tsx were developed to use the DRY (Don’t repeat yourself) principles. They get used several times, so it was important to make them reusable without having to change the code every unique instance.

On the other hand, there are functions that do very specific tasks like load_and_clean_job_data. It formats the data from the .csv file to be usable for the ML model.

Documentation Standards

Comments were used to explain important or long sections of code. Commit messages were consistent and descriptive. Git messages to each commit were used to track the changes in the code base. A ReadMe is attached to the GitHub project to provide an overview of the project and the usage instructions.

Maintainability

Separation of work: The client-server architecture separates between the frontend UI logic and the backend business logic allowing parallel programming.

Clear naming: All variables, functions, classes, and files were given distinctive and descriptive names for readability and understanding.

Configuration: Sensitive information and specific settings were managed using .env files and excluded from version control.

Errors: A lot of try-except blocks were used in the backend to handle potential problems during DB interactions, ML operations, and API processing. Common HTTP error responses were also addressed.

Scalability: While the technologies used do allow for significant scalability loading the data set in memory becomes a constraint. The caching system improves startup performance but does not address the runtime memory limits.

Iterative development: Core functionalities were added first. Secondary features and refinements were implemented afterwards.

Implementation of Key Functionalities:

Authentication & Profile management: On the front end the LoginForm and SignupForm components interact with the Firebase client SDK. AuthContext enables the persistence of the user state across all pages of the application. The backend uses get_current_user to verify the JWTs (JSON Web Tokens) (standard to safely transmit information between partiesas a JSON object) (auth0, n.d.) to ensure secure API access. User data is saved in Firestore. CRUD data is handled by FastAPI endpoints, they validate data using Pydantic models and update the user document in Firestore. Consistent profile creation in Firestore was a challenge for a user's first authentication, it was solved using a function called post_authentication.

Job Data Processing & Embedding Generation: The job data is stored in a CSV file and loaded in a Pandas DataFrame using the load_and_clean_job_data function. This process does data cleaning, handles missing values, and parses column data into data Python can use. A model from the sentence-transformers library is used to generate semantic embeddings. A big challenge was the long loading time caused by having to compute the embeddings of each job description on every launch. This problem became more apparent the more complex the model was. The solution to this was caching. By using the cache, embeddings are saved in a .pt file using torch.save(). Following startups load the embeddings using torch.load().

Job Matching Algorithm: This is the brain of the application. It implements a hybrid scoring system that uses several features like semantic_profile_weight, keyword_weight, experience_level_weight, education_semantic_weight, and education_presence_bonus. All these variables are given a weight bigger than 0 and smaller than 1. Each of them is multiplied with their corresponding score variable. main_s with semantic_profile_weight; keyword_s with

keyword_weight; experience_s with experience_level_weight; education_semantic_s with education_semantic_weight. education_presence_bonus is just an if statement chec to see if any eduacion exists to give a slight boost to the score, otherwise its value is considered 0. The score variables are calculated in different ways. Main_s uses the cosine similarity function and the ML model to be calculated. It is then multiplied with the weight to get the final semantic score. After all features are calculated the final_score is compared to a threshold, the min_score_threshold and if it surpasses it then it is recommended to the user.

This code shows the weights:

```
@app.get("/api/jobs/match", response_model=List[MatchedJob])

async def get_job_matches(
    current_user: Annotated[dict, Depends(get_current_user)],
    min_score_threshold: float = 0.6,
    semantic_profile_weight: float = 0.3,
    keyword_weight: float = 0.3,
    experience_level_weight: float = 0.25,
    education_semantic_weight: float = 0.1,
    education_presence_bonus: float = 0.02
):
```

This code shows the user's semantic vector:

```
user_profile_text_parts_main = []
if user_skills_list: user_profile_text_parts_main.append("Key skills: " + ", ".join(user_skills_list) + ".")
if user_experience_list_raw:
    exp_texts = [f"Professional experience as {exp.get('title', 'a role')}..." for exp in user_experience_list_raw if exp.get('title')]
    if exp_texts: user_profile_text_parts_main.append("Work history includes: " + " ".join(exp_texts))
user_text_for_main_embedding = " ".join(user_profile_text_parts_main).strip() or "General profile"
user_main_embedding = sentence_model.encode(user_text_for_main_embedding, convert_to_tensor=True).unsqueeze(0)
```

This code shows the cosine similarity coparison between the user embedding and job embedding:

```
all_main_semantic_scores_np = util.cos_sim(user_main_embedding, job_embeddings)[0].cpu().numpy()
for idx, job_row in jobs_df.iterrows():
    main_s = float(all_main_semantic_scores_np[idx])
```

This code shows the calculation of the final score:

```
final_score = (semantic_profile_weight * main_s) + (keyword_weight * keyword_s) + \
        (experience_level_weight * experience_s) + (education_semantic_weight * education_semantic_s) + edu_presence_b
```

This code shows the threshold filtering, how the user skills are matched to the job skills, a dictionary for a job listing match, and how a job is added to the matched_job_outputs:

```
if final_score >= min_score_threshold:
    user_skills_set_lower = set(s.lower() for s in user_skills_list)
    job_skills_set_lower = set(s.lower() for s in job_row.get('requiredSkills', []))
    matching_keywords_lower = user_skills_set_lower.intersection(job_skills_set_lower)
    matching_keywords_display = sorted([s for s in user_skills_list if s.lower() in        matching_keywords_lower])

    try:
        job_data_dict = job_row.to_dict()
        job_instance_data = {
            "id": str(job_data_dict.get('id', '')), "title": job_data_dict.get('title', 'N/A'),
            "company": job_data_dict.get('company'), "description": job_data_dict.get('description'),
            "requiredSkills": job_data_dict.get('requiredSkills'), "experience_level_required": job_data_dict.get('experience_level_required'),
            "matchScore": final_score,
            "matching_keywords": matching_keywords_display,
            "score_components": {
                "semantic_profile_score": main_s, "keyword_skill_score": keyword_s,
                "experience_level_score": experience_s, "education_semantic_score": education_semantic_s
            }
        }
        matched_jobs_output.append(MatchedJob(**job_instance_data))
    except Exception as val_err: print(f"[ERROR] Skipping job index {idx} due to Pydantic validation: {val_err}")
```

Integration and System Configuration:

Frontend-Backend integration

API communication: Next.js communicates with FastAPI through RESTful API calls. All data exchanges happened over HTTP.

Backend URL config: The frontend uses an environmental variable (frontend/.env.local) to specify the URL for the backend API (http://localhost:8000).

CORS (cross-origin resource sharing) config: CORS Middleware is used by FastAPI to allow requests from the front end and prevent CORS problems.

Authentication flow: The front-end handles login/signup directly using the Firebase client SDK. Once successfully authenticated the frontend gets a JWT for the user. For all the following requests to API endpoints, the front end includes the JWT (Authorization: Bearer <token>) in the HTTP header. The backend uses get_current_user to extract the token and verify it.

Firebase Integration

Frontend: The client SDK is initialized at frontend/src/lib/firebase.config.js. It is configured using the Firebase project configuration object which is gained through the Firebase console. This interaction enables authentication services from the Next.js application.

Backend: The Amin SDK is initialized at backend/main.py. It contains a service account key in the form of a JSON file. It is downloaded from the Firebase console and grants the backend privileges to interact with the Firebase services. The key is excluded from version control via .gitignore. The SDK provides access to auth and Firestore.client () (for database operations).

Backend Component Integration

ML model: The specific model is defined by the constant MODEL_NAME in main.py. It allows quick and easy switching between different models.

Cache configuration: The directory for job embeddings and the naming conventions for cache files are constant, which ensures that a new cache file is created for every new model.

The os module is used to check for exiting cache files and to create a cache directory if needed.

Environmental variables and secretes management

Backend: An .env file is used to store sensitive information or environment-specific configs. In this case, the only secret is the service account key.

Frontend: An .env.local file stores the NEXT_PUBLIC_API_URL.

User Interface Implementation:

Component-Based Architecture: The UI uses a component-based approach. Components that are needed on several pages are reusable, including modals (Modal.tsx), navigation (Navbar.tsx), and authentication (Login.TSX, Signup.tsx). Page-specific logic is part of App Router page components.

Styling: Tailwind is great when styling with productivity and functionality in mind. Due to its practical approach, you can style without ever leaving the HTML.

State Management

Local State: The useState hook from React was used for managing the local state of components, including form inputs, loading animations for API calls, and error and success messages.

Global state: AuthContext is used to manage the global auth state (current user, loading status). It makes it possible for components to change their behavior or rendering based on the user login status. One instance is the navbar rendering. When the user is not logged in the navbar does not include a logout button. When the user is logged in the button becomes available.

User Interaction and Feedback

Forms: Client-side validation provides feedback for required fields or formatting errors.

Loading states: Text-like loading and saving are used to show the system is processing the requests. Uses Boolean state variables.

Error/Success messages: Self-dismissing messages that show problems or successful actions the user does.

UX considerations

Minimizing effort: For instance, the user can add several skills and then save them together instead of saving each skill individually.

Clear labeling: All buttons are clearly labeled to show their intended purpose.

Smooth authentication: The login/signup window is a popup over the current page. It does not redirect to different pages.

Presentation: The profile page allows CRUD operation to create a proper profile and sends updates to the backend. The JobMatchesPage fetches and shows jobs ranked from the /api/jobs/match endpoint.

Testing Strategy:

Unit testing

Backend: pytest was used to validate functions and logical components such as calculate_match_score with various inputs and load_and_clean_job_data

Frontend: no formal unit testing was conducted; however, Jest of React Testing Library could be used to test individual React components.

Integration testing

Backend: Profile update endpoints were tested to ensure data was sent and received correctly with Firestore. Get_current_user dependency was tested to check if would reject invalid ID tokens.

Frontend-backend integration: The tests focused on making sure correct endpoints were called, auth tokens were correctly part of a header, request payloads from the front end matched the pydantic models, and information from the back end was represented correctly in the front end.

System Testing

The system testing was done by the developer following these test cases:

User registration, User login and logout, Profile creation, viewing job matches, Ensuring proper behavior of the database and the matching algorithm for different users

Model testing

Match review: Checking manually if the recommended job was actually the most relevant.

Parameter Tuning: Testing semantic_weight, keyword_weight, and, min_score_threshold to figure a sweet spot for balance.

User acceptance testing

The project was shared with classmates, peers, and older people to check the usability factor. The focus of the UAT was to see if older people, not closely connected to technology would be able to navigate the website easily.

Test Cases and Scenarios:

Case 1: Registration

Scenario: A user is trying to create a new account

Steps: Access the landing page, click sign up, and choose to either use email & password or Google authentication

Expected outcome: Account creation is successful, and the user is redirected to their dashboard.

Actual outcome: As expected

Case 2: A user tries registration with an existing account

Expected outcome: An error message pops up showing email is already in use

Actual outcome: As expected

Case 3: Editing skills, education experience

Expected outcomes: the user adds or removes the skills, education, experience and saves changes; the system gives signaling success.

Actual outcomes: If skills, education, and experience are saved after being added the system adds them to the database, otherwise they are wiped. The opposite of removing them.

Case 4: Checking job matches

Expected outcome: valid matches appear

Actual outcomes: If profile is set up no matches will show up, ranking of jobs differs based on weights.

Case 5: UI/UX

Expected outcome: the app works fine in different browsers, different screen sizes, mobile screens.

Actual outcome: Same as expected outcome from the developer's testing.

Bugs and Issues:

Bug/ Issue	How it was identified	Resolution	Impact
UI not adapting to mobile screens	While testing to see how the application would appear in smaller screens by using Inspect Element in the browser	Made sure to turn the navbar into a dropdown menu for smaller resolution widths	The application is fully usable on mobile web now
Matching Final score	During model testing I saw that the scores were too low even when the profile matched the job requirements	I decided to make the matching more specific by including the experience semantic score, the education semantic score, the education existence score boost and adjusting the weights to give out a more balanced result	The matching scores increased significantly when the profile vector matched the job vector
Sign in timeout	After trying to use google login or sign up the popup would close without the authentication finishing	The firebase SDK was configured with the previous version, so I updated it	The sign in worked flawlessly after the change
React Components not loading	React components would not load when testing for the frontend functionality	It was due to wrong paths and naming conventions, so I made sure to update the paths and use a uniform naming convention for all files	After fixing the issue all components would load up successfully

Performance evaluation:

Requirements	Assessment Method	Results
Performance	Load testing	Backend API responds in less than a second. Caching significantly lowers the job embeddings load time
Scalability	Stress testing	Performance suffers more the larger the dataset is
Usability	Usability & UAT	Clean UI, tested by peers and older people and proved to be easy to navigate
Accessibility	UI guidelines compliance	Contrast and responsive design were based on the WCAG
Reliability	System testing	All components work consistently
Portability	Browser testing	Works properly in all popular web browsers
Maintainability	Code review	Modular architecture, code documented in GitHub, uniform naming conventions
Security	Firebase AUTH & Firestore review	Uses Firebase ID Tokens for authentication, Firestore rules are enforced, and HTTPS is used

User Feedback and Acceptance:

Overall, all testers could easily use the application without needing any help. Criticism was given to the dashboard page. After a user successfully logs in, they are redirected to the dashboard page. The feedback said that it felt like an unnecessary step since the dashboard was very plain and did not provide any unique or essential information. As a result of this I revamped the dashboard page to make it more appealing. I added a section that displays the top matches from the job listings, a section detailing the profile completion progression, and a section that displays data about the total number of matches and the highest match score. From the user feedback I also added the edit education and experience feature. Before testing, once you added either one, you'd need to delete and re-enter them to make changes. After the update an edit button is available for both, to make any necessary changes.

Meeting Objectives and Requirements:

Objective	Evaluation	Outcome
Accurate job matching with AI & NLP	The algorithm works correctly and ranks jobs appropriately	Met
Profile Management	Implemented using CRUD by adopting a simple UI & Firebase for the backend	Met
Recommendation system	Integrated successfully using weights & a threshold	Met
Intuitive &responsive UI	Took advantage of Tailwind CSS	Met
Secure Authentication	JWT validation & Firestore rules ensure a high level of security	Met
Live job data	Not implemented; currently using a Kaggle dataset	Not met
Scalability	Not optimized for large datasets	Fell Short
Semantic analysis	Used a pre trained sentence-transformers model coupled with cosine similarity	Slightly exceeded
Transparency	Clear scoring formula & the project is available publicly in GitHub	Exceeded

Summary:

This chapter explains the core features of this project such as authentication, profile management, the matching algorithm, the sentence-transformers model, and semantic relationships. It goes into detail about the core logic of how the matching works, the development steps, why certain development decisions were made, and the testing phases. Testing included units, integration, system and UAT, confirming usability. Performance was increased by utilizing caching; however, the in-memory loading prevents the application from having high scalability. All in all, the system performs on par with expectations considering the scope.

Transitional section: With the application fully completed and evaluated, the next chapter presents future recommendations for the application such as features the scope did not allow,

possibilities for the evolution of the system, the main achievements of the project, and the lessons learnt.

CHAPTER 5: RECOMMENDATIONS AND CONCLUSION

Summary of Project Achievements:

Overall, the Ai Job Matcher application achieved its core objectives and requirements. It delivers a secure, user-friendly, and functioning system that uses a hybrid matching algorithm that combines semantic relationships with keyword, education, and experience scoring, thus meeting the base requirement of the project. Firebase provides a secure database via Firestore and protected authentication by using JWTs. The UI and UX are a product of Next.js and Tailwind CSS focusing on a clean and intuitive approach. Measures were taken to properly test the application and ensure its reliability, usability, and availability. Despite falling short considering scalability and real time job listing integration, the focus on AI and NLP integration is a valuable application of these technologies in the recruitment field. It shows their potential and how beneficial they can be in the job market.

Lessons Learned:

On the technical front there were three key lessons. Using a modular design separated the frontend and backend development and helped greatly with separating concerns and the ability to integrate parallel programming. Pre-trained models simplify semantic matching while providing accurate results. Training a model can be a fulfilling task however I found that would take much

time and much better hardware. Caching the job embeddings also was a great move to significantly lower load times and increase performance.

On the management perspective there were two important lessons. Using an Agile plan by developing iteratively, continuously testing and adjusting proved to be useful when fine-tuning the matching algorithm. The implementation of Firebase features was rapid, simple, and provided state-of-the-art security.

Several challenges occurred; however, the most noticeable ones include: the management of large data in memory, which highlights the importance of planning early about scalability; debugging React components because of inconsistent naming conventions led to a lot of time being used over small mistakes that could have been avoided by managing the code well from the beginning.

Evaluation of Objectives:

Objective	Evaluation	Status	Reasoning
Design system that analyzes skills, education, and experience using AI & NLP	Used semantic relationships, analysis and embeddings to process user input	Met	The models from sentence-transformers are well trained and efficient at processing data
Develop recommendation system using ML	Used a hybrid algorithm with semantic similarity, keywords, education and experience	Met	System recommends relevant matches and has adjustable weights for each variable
Develop a simple and intuitive UI	Used Next.js and Tailwind CSS, tested on several screen sizes and browsers	Met	User feedback was positive after using the app
Establish a secure system	Used Firebase Auth, Firestore & JWT authentication	Met	Endpoints were secured and data access is exclusive to authorized users
Evaluate system by testing and feedback	Used unit, integration, system, and UAT	Met	The tested features are functional, and feedback was mostly positive
Integrate live job data and	Not implemented	Not met	Out of scope, replaced with a Kaggle dataset
Scalability	Job data is loaded in memory causing the performance to drop with larger datasets	Partially met	Works well for small to medium sized datasets, needs refactoring for a production environment

Challenges Encountered:

Challenge	Impact	Solution
Long load times for embeddings	Significantly delayed API responses	I Implemented caching using torch.save() and torch.load()
Large datasets in memory	Limits scalability and lowers performance	It was accepted as a limitation and then considered as a feature for future improvements
Sign-in issues with Google	Authentication would time out	Firebase SDK configuration was updated
Errors when rendering React components	UI components would fail to load	I fixed file paths and incorrect naming conventions
Finding a balanced matching score	Irrelevant jobs were being recommended and good matches had lower than expected score	I adjusted scoring weights and added bonuses and penalties for education and experience
Rel-time job data	Negatively impacts the realism of the system	A static dataset is used
Time limitations and individual work	Restricted the overall advancements of the project	The core functionalities were given priority; agile principles were adopted

Recommendations for Future Work:

Real-time Job data addition: One of the major limitations of the project currently is the usage of a dataset to provide the job data. This also brings about the performance issue of having to load all the job data in memory. Future development should integrate live listings through APIs from major job boards like LinkedIn and Indeed.

Scalability improvement: As mentioned previously, the current system loads all job data in memory hindering performance with larger datasets. Implementing pagination, batched processing, or using vector databases like Pinecone should improve scalability.

User Feedback loop: Assuming the project goes to production a system could be implemented for the user to provide feedback on the matches which would allow the algorithm to adapt based on the user requirements. Future recommendations could use reinforcement learning.

Employer Portal: The project now only considers the job seeker as the user. A future feature could allow employees to upload opportunities themselves using the application

Resume Parsing and Suggestions: A future iteration of the app could parse resume information to fill in the profile automatically. It could also offer suggestions on what sections of the profile need improvement and how to improve them.

Mobile application: While the app works effectively on mobile browsers a dedicated mobile application could improve accessibility and increase engagement.

Advanced analytics: Features such as current job trends, most-matched industries and market demand could evolve the app from a job matching tool to a career assistant.

Potential Impact and Future Directions:

This project has the potential to improve the job-seeking experience for users who struggle to find roles that align with their skillset. Its AI and NLP capabilities make it possible to understand a user's qualification beyond keyword matching through the usage of semantic relationships of their skills, education, and experience to the job descriptions. Such an approach can reduce unemployment duration, it can improve career alignment, and it makes job searching accessible especially for new graduates or people transitioning career fields.

As mentioned above, if advanced analytics features are added the application could aim to become a career assistant. It could also help employees add their job opportunities directly to the site. This would cause the app to act as a job board.

Conclusion:

This project's core goal was to design a system that uses AI, ML, and NLP to create a powerful and accurate matching algorithm that takes in user skills, experience, education and job descriptions and recommends the best results to the user.

It adopts modular architecture with Next.js for the frontend, FastAPI for the backend, and Firebase for the authentication and the database. Its main component is the above-mentioned algorithm. It provides transparency towards the user by explaining the matching logic and by being an open-source project.

While the core components are functional this project serves as the baseline for a system that can have many additions in several areas. In conclusion, it is a good representation of how useful artificial intelligence can be during recruitment.

References

Amazon. (n.d.). *What is Natural Language Processing (NLP)?* Retrieved from aws: https://aws.amazon.com/what-is/nlp/

Atlassian. (n.d.). *What is the Agile methodology?* . Retrieved from Atlassian: https://www.atlassian.com/agile

auth0. (n.d.). *JSON Web Tokens*. Retrieved from auth0 Docs: https://auth0.com/docs/secure/tokens/json-web-tokens

Center, G. C. (n.d.). *MLOps: Continuous delivery and automation pipelines in machine learning*. Retrieved from Google Cloud: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

Coursera. (2024, June 13). *What Is a Skills Gap?* Retrieved from coursera: https://www.coursera.org/enterprise/articles/what-is-skills-gap

CSS, T. (n.d.). *Rapidly build modern websites without ever leaving your HTML.* Retrieved from tailwindcss: https://tailwindcss.com/

Face, H. (2025). *all-MiniLM-L6-v2*. Retrieved from hugginface: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Face, H. (2025). *all-mpnet-base-v2*. Retrieved from huggingface: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Face, H. (2025). *Sentence Transformers*. Retrieved from huggingface: https://huggingface.co/sentence-transformers

FastAPI. (n.d.). *FastAPI*. Retrieved from FastAPI: https://fastapi.tiangolo.com/

Fielding, R. T. (2000). *Representational State Transfer (REST)*. Retrieved from UCI: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Firebase. (n.d.). *Add the Firebase Admin SDK to your server*. Retrieved from Firebase: https://firebase.google.com/docs/admin/setup

Firebase. (n.d.). *Firebase*. Retrieved from Firebase: https://firebase.google.com/

Forum, W. E. (2025). *Future of Jobs Report 2025.* World Economic Forum .

GeeksForGeeks. (2024, Jul 24). *Zero-Shot vs One-Shot vs Few-Shot Learning*. Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/zero-shot-vs-one-shot-vs-few-shot-learning/

Google. (2025, April 16). *What is Machine Learning?* Retrieved from Google For Developers: https://developers.google.com/machine-learning/intro-to-ml/what-is-ml

Grinblat, F. (2024, October 2). *The Benefits of Semantic Search Over Keyword Matching in Resume Screening*. Retrieved from Brainner: https://www.brainner.ai/blog/article/the-benefits-of-semantic-search-over-keyword-matching-in-resume-screening

Henderson, R. (2025, April 7). *What Is an Applicant Tracking System (ATS)?* Retrieved from Jobscan: https://www.jobscan.co/blog/8-things-you-need-to-know-about-applicant-tracking-systems/

Jobscan. (2025). Retrieved from Jobscan: https://www.jobscan.co/

Kaggle. (n.d.). *How to Use Kaggle*. Retrieved from Kaggle: https://www.kaggle.com/docs/datasets

Ken Gu, R. S. (2021, September 8). *An Introduction to Semantic Matching Techniques in NLP and Computer Vision*. Retrieved from Medium: https://medium.com/georgian-impact-blog/an-introduction-to-semantic-matching-techniques-in-nlp-and-computer-vision-c22bf3cee8e9

Ouakili, O. E. (2025, March). *The Impact of Artificial Intelligence (AI) on Recruitment Process* . Retrieved from SCRIP: https://www.scirp.org/journal/paperinformation?paperid=140491

Pydantic. (n.d.). *Pydantic*. Retrieved from Pydantic: https://docs.pydantic.dev/latest/

Python. (n.d.). *re — Regular expression operations*. Retrieved from Python: https://docs.python.org/3/library/re.html

Python. (n.d.). *typing — Support for type hints*. Retrieved from Python: https://docs.python.org/3/library/typing.html

PyTorch documentation. (n.d.). Retrieved from PyTorch: https://docs.pytorch.org/docs/stable/index.html

Rehkopf, M. (n.d.). *Scrum Sprints: Everything You Need to Know*. Retrieved from Atlassian: https://www.atlassian.com/agile/scrum/sprints

Roland Rathelot, T. v.-Y. (2023, November). *Rethinking the skills gap* . Retrieved from IZA World of Labor: https://wol.iza.org/articles/rethinking-the-skills-gap/long

Samarth Gore, S. K. (2025, February). *A Review on Applicant Tracking System.* Retrieved from IRJET: https://www.irjet.net/archives/V12/i2/IRJET-V12I269.pdf

Singh, V. K. (2022, July). *VECTOR SPACE MODEL: AN INFORMATION RETRIEVAL SYSTEM.* Retrieved from ResearchGate: https://www.researchgate.net/publication/362060638_VECTOR_SPACE_MODEL_AN_INFORMATION_RETRIEVAL_SYSTEM

spaCy. (2025). *spaCy 101: Everything you need to know*. Retrieved from spaCy: https://spacy.io/usage/spacy-101

vmock. (n.d.). *Career Acceleration Platform*. Retrieved from vmock: https://www.vmock.com/

What is Semi-Structured Data? (n.d.). Retrieved from teradata.: https://www.teradata.com/glossary/what-is-semi-structured-data

What's in Next.js? (n.d.). Retrieved from NEXT.JS: https://nextjs.org/

Yanamaddi, L. G. (2023, March). *Enhancing Job Matching Accuracy: A Vector Search Approach for Resume-to-Job Description Alignment*. Retrieved from ResearchGate: https://www.researchgate.net/publication/388531365_Enhancing_Job_Matching_Accuracy_A_Vector_Search_Approach_for_Resume-to-Job_Description_Alignment